



Instituto Tecnológico de Mazatlán



**Comunicación I2C entre 4 Arduinos
con Arduino UNO R4 WiFi**

Carrera:

Ingeniería en Sistemas Computacionales

Materia:

Sistemas Programables

Profesor:

Miguel Barrón Hernández

Integrantes:

Ximena Armenta Dávila

Carlos Tadeo Ibarra Castañeda

Dulce Princesa Ulivarría González

1. Introducción

El bus I2C (Inter-Integrated Circuit) es un protocolo de comunicación serie que permite conectar varios dispositivos usando solamente dos líneas: SDA, por donde viajan los datos, y SCL, que lleva la señal de reloj. Uno de los dispositivos funciona como maestro y es el único que inicia la comunicación; los demás funcionan como esclavos y cada uno se identifica con una dirección única de 7 bits, de modo que pueden compartir las mismas dos líneas sin interferir entre sí.

En la presente práctica se armó un bus I2C con cuatro Arduino UNO R4 WiFi: un maestro y tres esclavos con las direcciones 0x08, 0x09 y 0x0A. El esclavo 1 enciende o apaga un LED según la orden que recibe, el esclavo 2 mueve un servomotor al ángulo que le envía el maestro y el esclavo 3 lee un potenciómetro y devuelve su valor cuando el maestro se lo pide. Cada 500 ms el maestro consulta el potenciómetro, convierte la lectura en un ángulo de 0° a 180° y lo reenvía al servo, mientras atiende desde el Monitor serie las órdenes para el LED. El circuito se diseñó primero en Tinkercad y después se armó de forma física.

2. Objetivos

Objetivo general:

- Comprender el funcionamiento del bus I2C mediante la comunicación entre un Arduino maestro y tres Arduinos esclavos que comparten las líneas SDA y SCL y se identifican con direcciones distintas.

Objetivos específicos:

- Armar el bus I2C uniendo las líneas SDA, SCL y GND de las cuatro placas, con resistencias pull-up de 4.7 kΩ.
- Asignar una dirección única a cada esclavo (0x08, 0x09 y 0x0A) con `Wire.begin()`.
- Enviar datos del maestro a un esclavo con `beginTransmission()`, `write()` y `endTransmission()`, y pedir datos a un esclavo con `requestFrom()` y `onRequest()`.
- Enviar la lectura del potenciómetro (0 a 1023) en dos bytes y reconstruirla en el maestro.
- Detectar cuando un esclavo no responde revisando el resultado de `endTransmission()` y usar `millis()` para que el maestro no se bloquee.

3. Herramientas y material utilizado

- 4 Arduino UNO R4 WiFi (un maestro y tres esclavos)
- 1 protoboard y cables de conexión (jumpers)
- 1 LED con su resistencia limitadora (220 Ω en Tinkercad, 470 Ω en el armado físico con el R4 WiFi)
- 1 servomotor
- 1 potenciómetro
- 2 resistencias de 4.7 kΩ (pull-up en SDA y SCL)
- 4 cables USB, uno por placa
- Arduino IDE 2 con las librerías Wire y Servo, y Monitor serie a 9600 baudios
- Tinkercad Circuits para el diagrama del circuito

4. Desarrollo / Metodología

Primero se armó el circuito en Tinkercad. Las líneas SDA y SCL de las cuatro placas se unieron en la protoboard, cada una con una resistencia pull-up de 4.7 kΩ hacia 5 V, y los GND de todas las placas se conectaron a una tierra común. En el

esclavo 1 se conectó el LED con su resistencia al pin 13, en el esclavo 2 la señal del servomotor al pin 9 y en el esclavo 3 el cursor del potenciómetro a la entrada analógica A0.

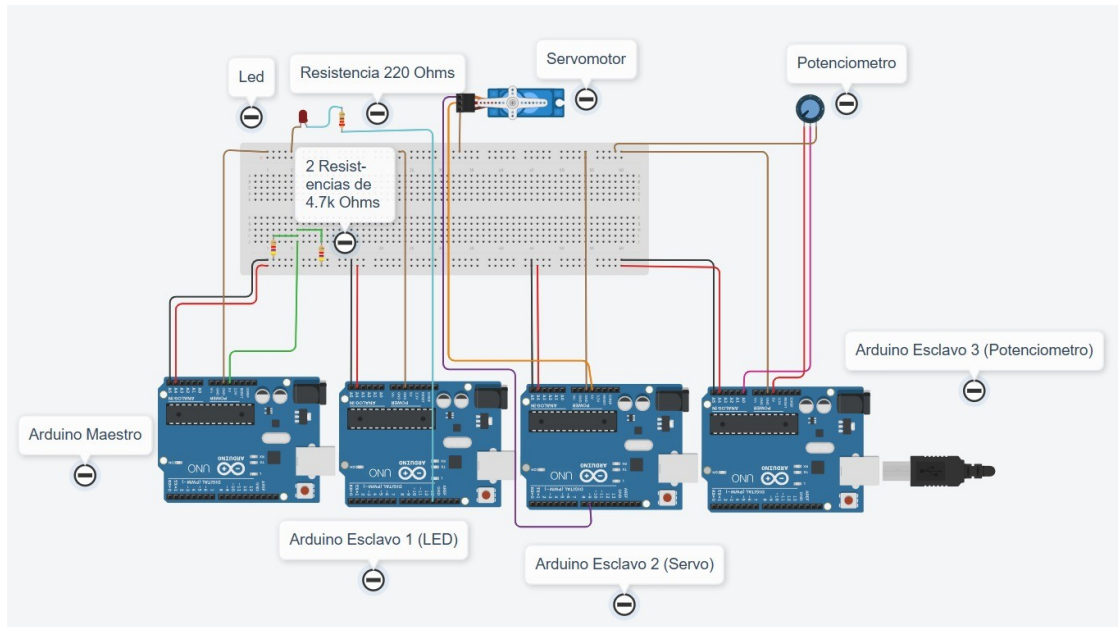


Figura 1. Diagrama del circuito en Tinkercad: Arduino maestro, esclavo 1 (LED), esclavo 2 (servo) y esclavo 3 (potenciómetro) sobre un mismo bus I2C.

La Tabla I resume la función y la dirección de cada placa dentro del bus.

Tabla I. Dirección y función de cada Arduino en el bus I2C.

Placa	Dirección	Periférico	Función
Maestro	—	Monitor serie	Inicia toda la comunicación, pide el potenciómetro cada 500 ms y envía órdenes al LED y al servo.
Esclavo 1	0x08	LED (pin 13)	Recibe un byte (1 o 0) y enciende o apaga el LED.
Esclavo 2	0x09	Servomotor (pin 9)	Recibe un ángulo de 0 a 180 y mueve el servomotor.
Esclavo 3	0x0A	Potenciómetro (A0)	Lee el potenciómetro y lo envía en dos bytes cuando el maestro lo pide.

Cada Arduino se programó con su propio sketch. El maestro (Maestro.ino) inicia el bus con `Wire.begin()` sin dirección. En cada vuelta del `loop()` revisa si llegó un 1 o un 0 por el Monitor serie y, si es así, se lo envía al esclavo 1. Además, usando `millis()`, cada 500 ms ejecuta `requestFrom(0x0A, 2)` para pedir dos bytes al esclavo 3, reconstruye el valor con `(alto << 8) | bajo`, lo convierte a ángulo con `map(valor, 0, 1023, 0, 180)` y lo envía al esclavo 2. Después de cada transmisión se revisa el valor que devuelve `endTransmission()`: si es distinto de 0, el esclavo no respondió y se muestra un aviso en el Monitor serie.

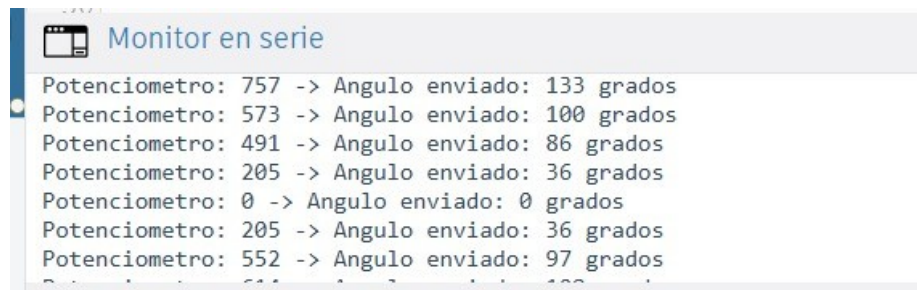
En los esclavos, la recepción se atiende con `Wire.onReceive()` y el envío con `Wire.onRequest()`. En el esclavo 2 el dato se lee una sola vez y se guarda en una variable; el servomotor se mueve dentro del `loop()` y no dentro de la rutina de recepción, para que esta termine lo más rápido posible. El esclavo 3 divide la lectura de 10 bits en dos bytes con `highByte()` y `lowByte()`.

Al pasar del simulador a las placas físicas UNO R4 WiFi se hicieron los siguientes ajustes:

- **Monitor serie:** en el R4 el puerto serie es USB nativo, por lo que al maestro se le agregó la línea `while ( ! Serial && millis() < 3000) { }` para no perder los primeros mensajes.
- **Resistencia del LED:** se usó 470 Ω en lugar de 220 Ω , porque los pines del R4 soportan menos corriente que los del R3.
- **Pull-ups:** las resistencias de 4.7 k Ω en SDA y SCL dejaron de ser opcionales, ya que las placas solas no traen pull-ups suficientemente fuertes.
- **Tierra y alimentación:** se unieron los GND de las cuatro placas, pero no los 5 V; cada placa se alimentó por su propio cable USB.

5. Evidencia del circuito y del monitor serie funcionando

A continuación se presenta la captura del Monitor serie del maestro, donde se muestra cada lectura del potenciómetro recibida desde el esclavo 3 y el ángulo que se envió al servomotor del esclavo 2.



```

Monitor en serie
Potenciómetro: 757 -> Angulo enviado: 133 grados
Potenciómetro: 573 -> Angulo enviado: 100 grados
Potenciómetro: 491 -> Angulo enviado: 86 grados
Potenciómetro: 205 -> Angulo enviado: 36 grados
Potenciómetro: 0 -> Angulo enviado: 0 grados
Potenciómetro: 205 -> Angulo enviado: 36 grados
Potenciómetro: 552 -> Angulo enviado: 97 grados

```

Figura 2. Monitor serie del maestro: valor del potenciómetro y ángulo enviado al servomotor.

Asimismo, se presenta la evidencia fotográfica del circuito físico armado con los cuatro Arduino UNO R4 WiFi, el LED encendido, el servomotor y el potenciómetro.

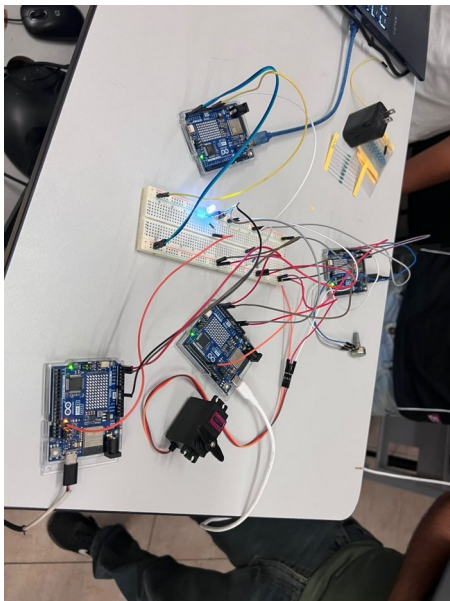


Figura 3. Vista general del armado físico con las cuatro placas.

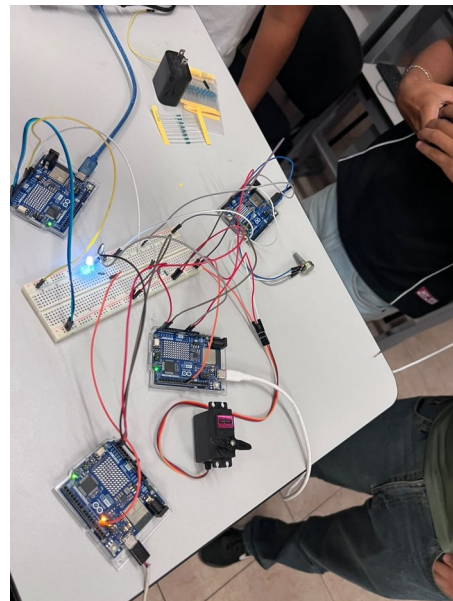


Figura 4. Otra vista del armado: LED encendido, servomotor y potenciómetro.

6. Análisis de resultados

Durante la prueba se giró el potenciómetro de un extremo a otro mientras el maestro mostraba las lecturas en el Monitor serie. La Tabla II reúne las lecturas capturadas en la Figura 2 y las compara con el valor exacto de la conversión $\text{valor} \times 180 / 1023$.

Tabla II. Lecturas del potenciómetro y ángulo enviado al servomotor.

Lectura	Potenciómetro (0–1023)	Cálculo exacto ($\text{valor} \times 180 / 1023$)	Ángulo enviado (°)
1	757	133.20	133
2	573	100.82	100
3	491	86.39	86
4	205	36.07	36
5	0	0.00	0
6	205	36.07	36
7	552	97.13	97

Observaciones sobre el comportamiento del sistema:

- Todos los ángulos enviados coinciden con la parte entera del cálculo exacto (por ejemplo, $757 \rightarrow 133.19 \rightarrow 133^\circ$), ya que `map()` trabaja con números enteros y descarta los decimales.
- El valor de 10 bits del potenciómetro llegó completo al maestro dividido en dos bytes: se registraron lecturas mayores a 255 (757, 573, 552), que no cabrían en un solo byte.
- Al llevar el potenciómetro a su extremo se obtuvo 0 y el servomotor se colocó en 0° ; al regresar a la misma posición se repitió la lectura $205 \rightarrow 36^\circ$, lo que muestra que la conversión es consistente.
- No apareció ningún mensaje de "no responde" en el Monitor serie, lo que confirma que los esclavos contestaron en sus direcciones y que el bus funcionó con las pull-ups de $4.7 \text{ k}\Omega$.
- El LED del esclavo 1 respondió a las órdenes 1 y 0 del Monitor serie sin interrumpir la lectura periódica del potenciómetro, gracias a que el maestro usa `millis()` en lugar de `delay()`.

Durante el armado se identificaron dos errores frecuentes: colocar ambas patas de la resistencia en la misma columna de la protoboard, lo que deja al LED sin resistencia (la corriente sube a unos 53.5 mA y el LED se quema), y leer varias veces el mismo dato de I2C dentro de una expresión, ya que el dato se consume en la primera lectura y las siguientes devuelven un valor inválido.

7. Conclusiones individuales

Ximena

Esta práctica me permitió entender que en el bus I2C el maestro es quien controla toda la comunicación: decide con qué esclavo habla, cuándo lo hace y si le pide o le manda datos, mientras que cada esclavo solo responde cuando lo llaman por su dirección. Me pareció muy práctico que para agregar un esclavo nuevo no se necesiten más pines, solo una dirección diferente.

También aprendí la importancia de que cada dirección sea única, porque si dos esclavos compartieran la misma ambos responderían al mismo tiempo y los datos se corromperían. Ver en el Monitor serie que las lecturas y los ángulos coincidían con el cálculo me confirmó que la comunicación entre las cuatro placas era correcta.

Carlos Tadeo

El desarrollo de esta práctica me ayudó a comprender cómo se envía un valor mayor a un byte por I2C. Como el potenciómetro entrega valores de 0 a 1023, fue necesario dividir la lectura en dos bytes con `highByte()` y `lowByte()` en el esclavo y volver a unirlos en el maestro; las lecturas de 757 o 573 que vimos en el Monitor serie demuestran que la reconstrucción funcionó.

Además, me pareció importante revisar el resultado de `endTransmission()`, porque permite saber cuándo un esclavo no contesta en lugar de que el sistema se quede congelado. También entendí por qué en el esclavo del servo el dato debe leerse una sola vez y guardarse en una variable, ya que cada lectura consume el dato del bus.

Dulce Princesa

Considero que el mayor aprendizaje de esta práctica fue pasar del simulador a las placas físicas. En Tinkercad el circuito funcionaba con el código original, pero con los Arduino UNO R4 WiFi tuvimos que esperar a que se abriera el puerto USB, usar una resistencia mayor para el LED, agregar las pull-ups de 4.7 kΩ y unir solamente las tierras, alimentando cada placa por su propio USB.

Adicionalmente, usar `millis()` en lugar de `delay()` en el maestro permitió que atendiera las órdenes del Monitor serie para el LED al mismo tiempo que consultaba el potenciómetro cada 500 ms. Esto me hizo ver que, al coordinar varias placas, es fundamental que ninguna tarea bloquee a las demás.

Referencias

[1] Arduino, "Arduino UNO R4 WiFi", documentación oficial. [En línea]. Disponible en: <https://docs.arduino.cc>

[2] Arduino, "Wire", referencia oficial del lenguaje Arduino. [En línea]. Disponible en: <https://docs.arduino.cc/language-reference/en/functions/communication/wire/>

[3] Arduino, "Servo", referencia de la librería Servo. [En línea]. Disponible en: <https://docs.arduino.cc/libraries/servo/>